

Fase 5 — Automatizaciones y copiloto

Estado

Implementado y compilado. No ejecutado (sin credenciales de Supabase ni programador de cron externo en este entorno). Preparado para conectarse a un scheduler real (Vercel Cron, GitHub Actions, cron del servidor, etc.).

Qué reutilicé

- `reglas_estancamiento` (tabla existente + regla sembrada en Fase 1): el motor no inventa umbrales, lee la configuración de negocio de esa tabla.
- `tareas_crm`, `notas_crm`, `historial_scoring_oportunidad`, `oportunidades`: las acciones automáticas escriben en las tablas canónicas, sin estructuras paralelas.
- **Service role helper** (`getServiceRoleClient`) para la ejecución del cron.
- **Copiloto determinista** (`src/lib/leadcenter/copiloto.ts`), ya entregado en la Fase 4, es la contraparte “en vivo” del motor de estancamiento.

Qué extendí y por qué

- `fn_evaluar_estancamiento(p_limite)` (nueva migración `20260829120700_leadcenter_fn_estancamiento.sql`): recorre oportunidades activas, calcula horas sin actividad desde el último cambio de etapa y aplica las acciones de la regla (crear tarea, reducir score, mover a nurturing, escalar a humano). Es **idempotente**: mientras exista una tarea automática `automatica_estancamiento` pendiente para la oportunidad, no la vuelve a procesar. Así el cron puede correr con la frecuencia que se quiera sin duplicar efectos.
- `/api/cron/automatizaciones`: endpoint protegido por `CRON_SECRET` (cabecera `Authorization: Bearer` o `?secret=`). Si el secreto no está configurado, el endpoint se autodeshabilita (503) para evitar ejecuciones anónimas. Invoca la RPC con service role.

Principio de diseño del copiloto/automatización

- El **copiloto nunca actúa solo**: sugiere y el asesor decide; cada decisión se audita como nota interna.
- La **automatización de estancamiento** sí realiza acciones acotadas y seguras (crear tarea, ajustar score/temperatura, dejar constancia), pero **nunca** transfiere datos, nunca borra y nunca contacta a la persona por su cuenta: cualquier envío real queda para el flujo de comunicaciones (pendiente).

Archivos

- `supabase/migrations/20260829120700_leadcenter_fn_estancamiento.sql`

- `app/api/cron/automatizaciones/route.ts`

Migraciones

- `fn_evaluar_estancamiento(integer)` — SECURITY DEFINER, idempotente.

Pruebas

- Compilación del endpoint con `next build` (Fase 6).
- No ejecutado contra BD real por falta de credenciales.
- Idempotencia razonada y garantizada por la guarda de tarea pendiente.

Cómo se conecta a producción (pendiente de infraestructura)

1. Definir `CRON_SECRET` en el entorno.
2. Programar una llamada periódica (p. ej. cada hora) a `POST /api/cron/automatizaciones` con `Authorization: Bearer $CRON_SECRET`.
3. (Opcional) Ajustar `p_limite` o la frecuencia según volumen.

Riesgos

- El cálculo de “tiempo en etapa” usa `historial_etapas_oportunidad`; si una oportunidad no tiene historial, cae a `actualizado_en`.
- La reducción de score usa un decremento fijo (−5); si se desea que dependa de la regla, se añadiría una columna `puntos_a_restar` a `reglas_estancamiento` (no se hizo para no alterar el esquema).

Decisión para continuar

Avanzar a la Fase 6: verificación (build), documentación de operación y control de versiones.